

AgroElevate — Complete Viva Study Guide

Project: AgroElevate (Agricultural Marketplace Platform)

Platform: Android Mobile Application

Framework: React Native + Expo SDK 54

Language: TypeScript

Routing: Expo Router

Backend: Supabase (PostgreSQL, Auth, Storage, Edge Functions)

Payments: Razorpay

Package: com.agronex.farmconnect

SECTION 1 — What is AgroElevate?

1.1 Problem Statement

Indian agriculture suffers from fragmented supply chains. Farmers often sell produce at mandis through middlemen who capture most of the profit margin. Traders add value through logistics but may not compensate original growers when crops are resold. Industrial buyers need bulk supply but lack transparent pricing. End customers want fresh produce but cannot verify farmer origin.

AgroElevate solves this by providing a **digital agricultural marketplace** where Farmers, Traders (Middlemen), Industrialists, Customers, and Admins interact on one platform with **wallet-based payments, order tracking**, and an automatic **12.5% royalty** paid to the original farmer when a trader resells produce at a profit.

1.2 Why This Project Exists

- **Farmer exploitation:** Traditional mandi systems give farmers low prices while intermediaries earn high margins.
- **Middlemen problems:** Traders are necessary for aggregation and distribution but profit sharing is opaque.
- **Industrialist issues:** Bulk buyers need reliable suppliers, price visibility, and traceability.
- **Need for digital marketplace:** Smartphones are widespread even in rural India; a mobile app bridges farmers and buyers directly.

1.3 Objectives

1. Build a multi-role Android marketplace connecting all agricultural stakeholders.
2. Enable farmers to list crops with images, pricing, and categories.
3. Implement secure authentication and role-based dashboards.
4. Provide wallet system with Razorpay top-up for purchases.
5. Automate royalty distribution (12.5% of profit) to original farmers on resale via `checkout_order` RPC.
6. Offer market intelligence (live prices, MSP, forecasts) for informed decisions.
7. Admin panel for user approval, wallet management, and platform analytics.

1.4 Motivation

Fair trade in agriculture. Technology can return value to the person who grows the food. AgroElevate encodes royalty logic in the database so it cannot be bypassed at checkout.

1.5 Expected Outcome

A production-ready Android APK/AAB that demonstrates end-to-end commerce: register → list/buy → pay → royalty credit → order tracking.

1.6 Advantages

- Direct farmer-to-market access
- Transparent royalty on resale
- Single app for 5 roles
- Real Razorpay integration for wallet top-up
- Supabase backend (scalable PostgreSQL)
- Cross-platform codebase (React Native)
- Market intelligence module

1.7 Limitations

- Email/password auth (not OTP in current build)
- Royalty only on profitable resale (profit must be positive)
- Wallet required before purchase (no COD)
- Image storage uses product metadata / category images (not always custom upload in listing)
- FCM notifications configured but not fully wired in all flows
- Requires internet connectivity

1.8 Future Scope

- Hindi/regional language support
- Phone OTP authentication
- GPS-based mandi discovery
- Blockchain traceability for organic certification
- AI crop disease detection
- Logistics partner integration
- Government MSP auto-sync APIs

SECTION 2 — Technology Stack

2.1 JavaScript

JavaScript is the programming language of the web. It runs in browsers and, via Node.js, on servers. React Native uses JavaScript (and TypeScript) to describe mobile UI. Variables use `let` / `const`, functions are first-class, and async operations use Promises and `async/await`.

2.2 TypeScript

TypeScript = JavaScript + static types. Our project uses `.tsx` files (TypeScript + JSX). Benefits: catch errors at compile time, better IDE autocomplete, self-documenting interfaces like `UserData` in `AuthContext`.

2.3 Node.js & NPM

Node.js runs JavaScript outside the browser. **NPM** (Node Package Manager) installs dependencies listed in `package.json`. Commands: `npm install`, `npm start`, `npx expo start`.

2.4 React Native

Framework by Meta to build native mobile apps using React. Instead of `<div>`, we use `<View>`, `<Text>`, `<TouchableOpacity>`. React Native bridge communicates with native Android/iOS widgets. **Why chosen:** One codebase for Android and iOS, large ecosystem, fast development.

2.5 Expo SDK 54

Expo is a layer on React Native that handles build tooling, native modules, and cloud builds (EAS). SDK 54 maps to React Native 0.81.5 and React 19. **Why chosen:** Simplifies Android builds without manual Gradle setup for every library; EAS Build produces APK/AAB in cloud.

2.6 Expo Router

File-based routing: each file in `app/` becomes a screen route. `app/farmer/dashboard.tsx` → `/farmer/dashboard`. Layouts use `_layout.tsx`.

2.7 Metro Bundler

JavaScript bundler for React Native. Transforms TypeScript/JSX into a single bundle the app loads. Started via `expo start`.

2.8 Android Studio, Gradle, ADB

- **Android Studio:** IDE for native Android development and emulator.
- **Gradle:** Build system for Android (`android/build.gradle`). Compiles Java/Kotlin and packages APK.
- **ADB (Android Debug Bridge):** Command-line tool to install APK, view logs (`adb logcat`), debug devices.

2.9 EAS Build

Expo Application Services — cloud builds. Config in `eas.json`. `preview` profile builds APK; `production` builds AAB for Play Store.

2.10 APK vs AAB

- **APK:** Single installable file for direct distribution/testing.
- **AAB (Android App Bundle):** Upload format for Google Play; Play generates optimized APKs per device.

2.11 Native Android vs Cross-Platform

Native Android (Kotlin/Java) gives maximum performance and platform APIs. Cross-platform (React Native) shares ~90% code between platforms. AgroElevate uses React Native + Expo with native modules (Razorpay, AsyncStorage) where needed.

2.12 Supabase vs Firebase

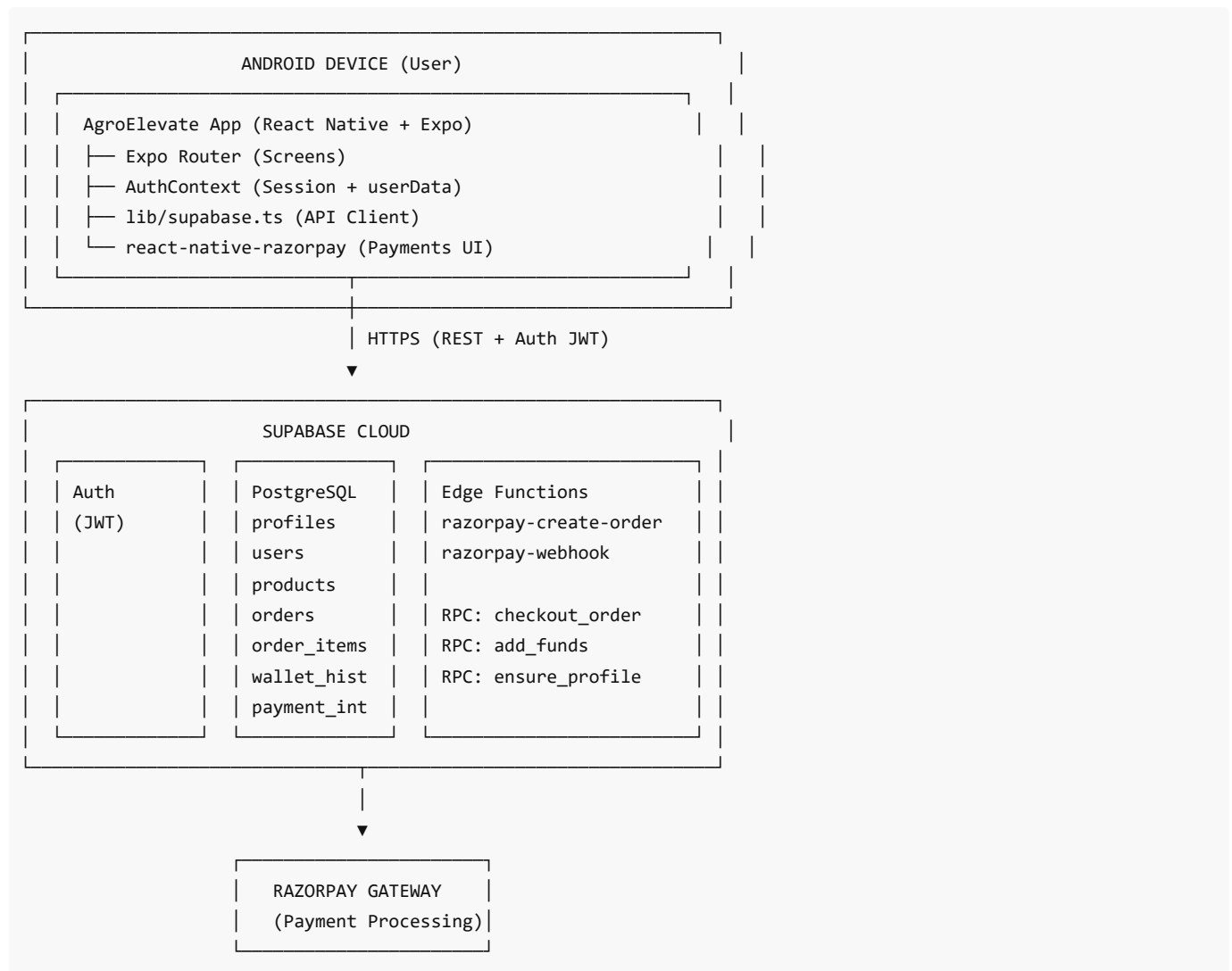
Supabase chosen for: PostgreSQL (relational data, joins, RPC), SQL migrations, Row Level Security, open-source, REST + realtime. Firebase was in early docs but production uses Supabase.

2.13 Razorpay

Indian payment gateway. Used for wallet top-up via Edge Function `razorpay-create-order` and `react-native-razorpay` native SDK on Android.

SECTION 3 — Project Architecture

3.1 High-Level Architecture



3.2 Data Flow — Login

1. User enters email/password → `supabase.auth.signInWithPassword`
2. Supabase returns JWT session → stored in `AsyncStorage`
3. `AuthContext` `onAuthStateChange` fires → `ensureUserRecords`
4. Creates/reads `profiles` and `users` rows
5. `index.tsx` reads `userData.role` → redirects to role dashboard

3.3 Data Flow — Purchase

1. User opens product → `products` table SELECT with seller profile join
2. User taps Buy → `checkout_order` RPC with cart `[[{id, qty}]]`
3. RPC: verify wallet balance → deduct buyer → create order → credit sellers → royalty to original farmer → update stock
4. App refreshes wallet via `refreshUserData`

3.4 Data Flow — Wallet Top-Up

1. User enters amount → `supabase.functions.invoke('razorpay-create-order')`
2. Edge function creates Razorpay order + `payment_intents` row
3. Native Razorpay checkout opens
4. On success, webhook marks intent paid → `add_funds` credits wallet
5. App polls `payment_intents.status` until `paid`

SECTION 4 — Folder Structure

4.1 Root Structure

```
app_agronex/
├─ frontend/      # React Native Expo app (main deliverable)
├─ backend/       # Legacy Python server + SQL schemas
├─ scripts/       # Build & guide generators
└─ README.md
```

4.2 frontend/app/ — Expo Router Screens

Path	Purpose
<code>index.tsx</code>	Entry: auth gate + role redirect
<code>_layout.tsx</code>	Root Stack navigator + AuthProvider
<code>auth/login.tsx</code>	Email/password login
<code>auth/register.tsx</code>	Registration with role selection
<code>farmer/*</code>	Farmer tabs: dashboard, marketplace, intelligence, orders, wallet, profile
<code>trader/*</code>	Trader screens (same pattern)
<code>customer/*</code>	Customer screens
<code>industrialist/*</code>	Industrialist screens
<code>admin/*</code>	Admin panel screens
<code>crop-details.tsx</code>	Shared product detail + buy flow

4.3 frontend/contexts/

`AuthContext.tsx` — Global authentication state: session, user, userData, signOut, refreshUserData, provisioning.

4.4 frontend/lib/

| File | Purpose |

|-----|-----|

| `supabase.ts` | Supabase client with AsyncStorage session |

| `walletApi.ts` | Balance & history queries |

| `ordersApi.ts` | Fetch buyer/seller orders |

| `adminApi.ts` | Admin stats, user management |

| `commerceMeta.ts` | Royalty metadata JSON in product description |

| `razorpayWallet.ts` | Top-up order + payment polling |

| `razorpayCheckout.native.ts` | Native Razorpay SDK wrapper |

| `roleUtils.ts` | Role normalization (middleman → trader) |

| `aiApi.ts` | Crop pricing assistant API |

4.5 frontend/components/

Reusable UI: `ErrorBoundary`, `ScreenPrimitives`, Intelligence components, `AdminScreenHeader`.

4.6 How Expo Router Works

File `app/farmer/_layout.tsx` defines Tab navigator. Hidden screens use `href: null`. Dynamic routes use `[id].tsx`. `useRouter()` for navigation, `useLocalSearchParams()` for query params.

SECTION 5 — Authentication

5.1 Login Flow

`login.tsx` calls `supabase.auth.signInWithPassword({ email, password })`. On success, navigates to `/` where `index.tsx` routes by role.

5.2 Registration

`register.tsx` calls `supabase.auth.signUp` with `options.data`: name, role, address, phone, bank_account. Role stored in auth user metadata.

5.3 JWT & Sessions

Supabase Auth issues **JWT** (JSON Web Token) containing user id and claims. Access token sent in Authorization header. **Refresh token** auto-renewed (`autoRefreshToken: true` in `supabase.ts`).

5.4 AsyncStorage

Key-value storage on device. Supabase auth persists session here. `AuthContext` also caches `userData` JSON for faster startup.

5.5 AuthContext Functions

- `refreshUserData`: `SELECT from users table by uid`

- `ensureUserRecords` : Creates `profiles` + calls `ensure_profile_from_auth` RPC for `users` `wallet` row
- `signOut` : Global sign out + clear `AsyncStorage`
- `onAuthStateChange` : Listens for `SIGNED_IN`, `SIGNED_OUT` events

5.6 Role-Based Routing

Roles: farmer, trader, customer, industrialist, admin. `index.tsx` switch statement maps role → dashboard path. Protected: no session → `/auth/login` .

SECTION 6 — Database Design

6.1 PostgreSQL

Open-source relational database. Tables, foreign keys, transactions, stored procedures (RPC). ACID compliant — critical for wallet operations.

6.2 Core Tables

profiles

Linked to Supabase Auth (`id` = `auth.users.id`). Columns: email, name, role, address, phone, bank_account, approved, suspended, created_at. Admin uses this for user management.

users

Wallet mirror table. Columns: uid (PK), phoneNumber, role, name, address, bankUPI, walletBalance, approved, createdAt. Used for balance and checkout.

products

Marketplace listings. Columns: id, seller_id, name, crop_type, quantity, unit, price_per_unit, description (JSON commerce meta), image_url, created_at.

orders

id, buyerId, buyerName, buyerRole, totalAmount, shippingAddress, status (pending/accepted/shipped/delivered), createdAt, updatedAt.

order_items

id, orderId, cropId/productId, farmerId/sellerId, cropName, quantity, unit, pricePerUnit, totalPrice, originalFarmerId.

wallet_history

id, userId, type, amount, orderId, description, createdAt. Types: debit, credit, royalty, add_funds, sale_income, royalty_income, purchase, etc.

payment_intents

Tracks Razorpay top-up: id, user_id, amount, status (pending/paid/failed), razorpay_order_id, receipt_number.

transactions

Platform transaction log for sales analytics.

notifications

userId, title, message, read, createdAt.

6.3 Keys & Relationships

- **Primary Key (PK):** Unique row identifier (UUID or TEXT uid)
- **Foreign Key (FK):** References another table (order_items.orderId → orders.id)
- **ON DELETE CASCADE:** Deleting order removes its items

6.4 snake_case vs camelCase

PostgreSQL convention: snake_case (price_per_unit). Legacy tables use camelCase quoted ("walletBalance"). Frontend normalizes both in API helpers.

6.5 RLS (Row Level Security)

Supabase feature to restrict rows per user. Schema.sql shows RLS disabled for migration; production should enable policies per role.

6.6 RPC Functions

- add_funds : Credit wallet + history
- create_order / checkout_order : Atomic purchase + royalty
- ensure_profile_from_auth : Sync auth user to profiles/users

SECTION 7 — Supabase

Supabase Client (lib/supabase.ts): createClient(url, anonKey, { auth: { storage: AsyncStorage } }) .

Queries: .from('table').select().eq().insert().update().delete()

RPC: .rpc('function_name', { params }) calls PostgreSQL functions.

Edge Functions: Serverless Deno functions at supabase.functions.invoke('name') .

Storage: Bucket for images (if configured).

Realtime: WebSocket subscriptions (optional for live prices).

SECTION 8 — Marketplace

farmer/marketplace.tsx loads products where quantity > 0, joins seller profile.

Search: Client-side filter on name. **Categories:** Grains, Vegetables, Fruits, Pulses, Raw Materials.

Farmer view: Hides trader relistings (isRelisted). **Trader/Customer:** See all.

Product details: crop-details.tsx — quantity input, total price, Buy Now → checkout_order RPC.

SECTION 9 — Wallet System

Balance from users.walletBalance . History from wallet_history .

Credits: add_funds, sale_income, royalty_income, credit, royalty, refund

Debits: debit, purchase, withdrawal

Top-up: Razorpay → payment_intents → add_funds

Admin credit: adminCreditWallet calls add_funds with payment_method 'admin_credit'

SECTION 10 — Payment System (Razorpay)

1. createWalletTopUpOrder(amount) → Edge Function
2. Returns key_id, order_id, amount_paise, intent_id
3. openRazorpayCheckout opens native payment UI
4. Webhook verifies signature → updates payment_intents → add_funds
5. pollWalletAfterPayment polls every 2s, max 30 attempts

Security: Secret keys only in Edge Functions, never in app. Anon key is public but RLS/RPC protects data.

SECTION 11 — Orders

ordersApi.fetchUserOrders merges buyer orders + seller orders (if seller role).

Statuses: pending → accepted → shipped → delivered.

Seller identified via order_items.farmerId or sellerId.

SECTION 12 — Royalty System

Commerce Meta stored as JSON in products.description :

```
{
  "product_kind": "trader_relist",
  "original_farmer_id": "uuid",
  "current_owner_id": "uuid",
  "ownership_chain": [...],
  "royalty_percent": 12.5,
  "purchase_price_per_unit": 50
}
```

Farmer listing: buildFarmerListingMeta(farmerId) sets product_kind = raw_farmer.

Trader relist: buildRelistMeta after purchasing from farmer.

checkout_order logic: For each item, if originalFarmerId and profit > 0: royalty = profit × qty × 12.5%. Seller gets totalPrice - royalty. Original farmer wallet credited.

SECTION 13 — Admin Panel

Screens: dashboard (stats), approve-farmers, manage-users, transactions, wallet-management, platform-settings.

fetchPlatformStats : total users, farmers, pending approvals, orders, volume.

Actions: approve/suspend users, admin wallet credit.

SECTION 14 — Android Development Basics

Activity: Single screen in native Android (React Native uses one MainActivity).

Manifest: `AndroidManifest.xml` — permissions, package name, activities.

Permissions: CAMERA, LOCATION, STORAGE declared in `app.json`.

Build Types: debug (dev) vs release (signed, optimized).

Keystore: Cryptographic key to sign release APK/AAB.

Emulator: Virtual device in Android Studio for testing.

SDK: Android API levels (`compileSdk`, `minSdk`).

NDK: Native Development Kit for C/C++ modules (Razorpay native bridge).

SECTION 15 — React Native Concepts

Components: Functions returning JSX. **Props:** Read-only inputs. **State:** `useState` for local data.

useEffect: Side effects on mount/dependency change. **useFocusEffect:** Refetch when screen focused.

Context API: `AuthContext` shares auth globally. **FlatList:** Efficient long lists. **ScrollView:** Scrollable content.

Performance: `flatListPerfProps`, memoization, avoid inline functions in large lists.

SECTION 16 — Expo

Expo Go: Dev client for quick testing (limited native modules).

Prebuild: `npx expo prebuild` generates `android/` `ios/` folders.

run:android: `npx expo run:android` builds and installs on device/emulator.

EAS Build: Cloud CI for APK/AAB. **OTA:** Expo Updates for JS bundle updates without store.

SECTION 17 — Libraries

- **@expo/vector-icons (Ionicons):** Icons throughout UI
- **react-native-svg:** Charts in intelligence module
- **react-native-screens:** Native screen containers for navigation
- **react-native-gesture-handler:** Touch gestures
- **react-native-reanimated:** Smooth animations
- **react-native-webview:** Embedded web content in intelligence maps
- **expo-image:** Optimized image loading
- **@react-native-async-storage/async-storage:** Session + cache persistence
- **react-native-razorpay:** Razorpay payment checkout
- **@supabase/supabase-js:** Backend client

- **expo-router:** File-based navigation
- **expo-location:** GPS for market intelligence
- **react-native-gifted-charts:** Dashboard analytics charts
- **expo-image-picker:** Crop photo upload

SECTION 18 — APIs

- Supabase REST: `https://<project>.supabase.co/rest/v1/`
- Supabase Auth: `/auth/v1/`
- RPC: POST `/rest/v1/rpc/checkout_order`
- Edge Functions: `/functions/v1/razorpay-create-order`
- Env vars: EXPO_PUBLIC_SUPABASE_URL, EXPO_PUBLIC_SUPABASE_ANON_KEY

SECTION 19 — Production

Testing: runtime-qa.mjs verifies RPC exists. Manual QA on emulator/device.

Build: `eas build --profile preview` → APK. `production` → AAB.

Secrets: Never commit Razorpay secret; use EAS secrets / Supabase Edge env.

Play Store: Upload AAB, fill listing, content rating, privacy policy.

SECTION 20 — Security

- JWT in Authorization header
- RPC SECURITY DEFINER runs with elevated privileges — must validate user inside function
- SQL injection prevented by parameterized queries
- XSS: React escapes text by default
- Payment verification on server webhook, not client alone

SECTION 21 — Complete User Flows

21.1 Farmer Flow

Login → Dashboard (analytics) → Add Crop (image, AI pricing) → INSERT products → Marketplace browse → Receive order notification → Wallet shows sale_income + royalty_income → Profile/Logout

21.2 Trader Flow

Login → Marketplace buy farmer crop → checkout_order → My purchases in orders → Relist at higher price (trader_relist meta) → Customer buys → Royalty auto-paid to farmer → Trader keeps profit minus royalty

21.3 Customer Flow

Login → Browse marketplace → crop-details → Buy → Wallet debited → Track order status

21.4 Industrialist Flow

Same as trader with bulk quantities; intelligence module for market data

21.5 Admin Flow

Login → Dashboard stats → Approve farmers → Manage/suspend users → Credit wallets → View transactions

SECTION 22 — Key Code Walkthrough

index.tsx — Role Router

Waits for loading=false. No user → /auth/login. Has userData → switch(role) → dashboard.

AuthContext — ensureUserRecords

Reads user_metadata.role, inserts profiles if missing, calls ensure_profile_from_auth RPC, refreshUserData from users table.

crop-details.tsx — handleBuy

Validates qty, builds cart [{id, qty}], calls checkout_order RPC, refreshUserData, navigates to orders.

commerceMeta.ts

parseCommerceMeta parses JSON from description. buildFarmerListingMeta / buildRelistMeta create royalty tracking metadata.

wallet.tsx — handleAddFunds

createWalletTopUpOrder → openRazorpayCheckout → pollWalletAfterPayment → refresh balance.

SECTION 23 — Expected Viva Questions (500 Total)

23.1 — Basic Questions (100 Questions)

Q1. What is AgroElevate?

Answer: A React Native Android agricultural marketplace connecting farmers, traders, customers, industrialists, and admins with wallet payments and automatic farmer royalty on resale.

Q2. What framework did you use?

Answer: React Native with Expo SDK 54 and TypeScript.

Q3. What is your backend?

Answer: Supabase — PostgreSQL database, authentication, edge functions, and RPC stored procedures.

Q4. How many user roles exist?

Answer: Five: Farmer, Trader, Customer, Industrialist, Admin.

Q5. What is React Native?

Answer: A cross-platform framework to build mobile apps using JavaScript/React that renders native UI components.

Q6. What is Expo?

Answer: A toolchain and platform on top of React Native for easier development, native module access, and cloud builds (EAS).

Q7. What is TypeScript?

Answer: A typed superset of JavaScript that compiles to JavaScript, catching errors at compile time.

Q8. What database do you use?

Answer: PostgreSQL hosted on Supabase.

Q9. What payment gateway?

Answer: Razorpay for wallet top-up.

Q10. What is the royalty percentage?

Answer: 12.5% of profit per unit on trader resale, paid to the original farmer.

Q11. What is JWT?

Answer: JSON Web Token — a signed token proving authentication, issued by Supabase Auth.

Q12. What is AsyncStorage?

Answer: Local persistent key-value storage on the device used for session caching.

Q13. What is Expo Router?

Answer: File-based routing where files in app/ directory map to navigation routes.

Q14. What is an APK?

Answer: Android Package — installable application file.

Q15. What is Gradle?

Answer: Android build automation tool that compiles and packages the app.

Q16. What is Metro?

Answer: The JavaScript bundler for React Native.

Q17. What is Supabase Auth?

Answer: Authentication service supporting email/password, issuing sessions and JWTs.

Q18. What is a Primary Key?

Answer: A column that uniquely identifies each row in a table.

Q19. What is a Foreign Key?

Answer: A column referencing a primary key in another table to enforce relationships.

Q20. What is RPC in Supabase?

Answer: Remote Procedure Call — invoking PostgreSQL functions from the client via API.

Q21. What is EAS Build?

Answer: Expo Application Services cloud build system for generating APK/AAB.

Q22. What is the package name?

Answer: com.agronex.farmconnect

Q23. What problem does royalty solve?

Answer: Ensures original farmers benefit when traders resell their produce at higher prices.

Q24. What is wallet balance stored in?

Answer: users.walletBalance column in PostgreSQL.

Q25. What is checkout_order?

Answer: Supabase RPC that atomically processes purchase, payments, royalties, and stock updates.

Q26. What is AgroElevate?

Answer: A React Native Android agricultural marketplace connecting farmers, traders, customers, industrialists, and admins with wallet payments and automatic farmer royalty on resale.

Q27. What framework did you use?

Answer: React Native with Expo SDK 54 and TypeScript.

Q28. What is your backend?

Answer: Supabase — PostgreSQL database, authentication, edge functions, and RPC stored procedures.

Q29. How many user roles exist?

Answer: Five: Farmer, Trader, Customer, Industrialist, Admin.

Q30. What is React Native?

Answer: A cross-platform framework to build mobile apps using JavaScript/React that renders native UI components.

Q31. What is Expo?

Answer: A toolchain and platform on top of React Native for easier development, native module access, and cloud builds (EAS).

Q32. What is TypeScript?

Answer: A typed superset of JavaScript that compiles to JavaScript, catching errors at compile time.

Q33. What database do you use?

Answer: PostgreSQL hosted on Supabase.

Q34. What payment gateway?

Answer: Razorpay for wallet top-up.

Q35. What is the royalty percentage?

Answer: 12.5% of profit per unit on trader resale, paid to the original farmer.

Q36. What is JWT?

Answer: JSON Web Token — a signed token proving authentication, issued by Supabase Auth.

Q37. What is AsyncStorage?

Answer: Local persistent key-value storage on the device used for session caching.

Q38. What is Expo Router?

Answer: File-based routing where files in app/ directory map to navigation routes.

Q39. What is an APK?

Answer: Android Package — installable application file.

Q40. What is Gradle?

Answer: Android build automation tool that compiles and packages the app.

Q41. What is Metro?

Answer: The JavaScript bundler for React Native.

Q42. What is Supabase Auth?

Answer: Authentication service supporting email/password, issuing sessions and JWTs.

Q43. What is a Primary Key?

Answer: A column that uniquely identifies each row in a table.

Q44. What is a Foreign Key?

Answer: A column referencing a primary key in another table to enforce relationships.

Q45. What is RPC in Supabase?

Answer: Remote Procedure Call — invoking PostgreSQL functions from the client via API.

Q46. What is EAS Build?

Answer: Expo Application Services cloud build system for generating APK/AAB.

Q47. What is the package name?

Answer: com.agronex.farmconnect

Q48. What problem does royalty solve?

Answer: Ensures original farmers benefit when traders resell their produce at higher prices.

Q49. What is wallet balance stored in?

Answer: users.walletBalance column in PostgreSQL.

Q50. What is checkout_order?

Answer: Supabase RPC that atomically processes purchase, payments, royalties, and stock updates.

Q51. What is AgroElevate?

Answer: A React Native Android agricultural marketplace connecting farmers, traders, customers, industrialists, and admins with wallet payments and automatic farmer royalty on resale.

Q52. What framework did you use?

Answer: React Native with Expo SDK 54 and TypeScript.

Q53. What is your backend?

Answer: Supabase — PostgreSQL database, authentication, edge functions, and RPC stored procedures.

Q54. How many user roles exist?

Answer: Five: Farmer, Trader, Customer, Industrialist, Admin.

Q55. What is React Native?

Answer: A cross-platform framework to build mobile apps using JavaScript/React that renders native UI components.

Q56. What is Expo?

Answer: A toolchain and platform on top of React Native for easier development, native module access, and cloud builds (EAS).

Q57. What is TypeScript?

Answer: A typed superset of JavaScript that compiles to JavaScript, catching errors at compile time.

Q58. What database do you use?

Answer: PostgreSQL hosted on Supabase.

Q59. What payment gateway?

Answer: Razorpay for wallet top-up.

Q60. What is the royalty percentage?

Answer: 12.5% of profit per unit on trader resale, paid to the original farmer.

Q61. What is JWT?

Answer: JSON Web Token — a signed token proving authentication, issued by Supabase Auth.

Q62. What is AsyncStorage?

Answer: Local persistent key-value storage on the device used for session caching.

Q63. What is Expo Router?

Answer: File-based routing where files in app/ directory map to navigation routes.

Q64. What is an APK?

Answer: Android Package — installable application file.

Q65. What is Gradle?

Answer: Android build automation tool that compiles and packages the app.

Q66. What is Metro?

Answer: The JavaScript bundler for React Native.

Q67. What is Supabase Auth?

Answer: Authentication service supporting email/password, issuing sessions and JWTs.

Q68. What is a Primary Key?

Answer: A column that uniquely identifies each row in a table.

Q69. What is a Foreign Key?

Answer: A column referencing a primary key in another table to enforce relationships.

Q70. What is RPC in Supabase?

Answer: Remote Procedure Call — invoking PostgreSQL functions from the client via API.

Q71. What is EAS Build?

Answer: Expo Application Services cloud build system for generating APK/AAB.

Q72. What is the package name?

Answer: com.agronex.farmconnect

Q73. What problem does royalty solve?

Answer: Ensures original farmers benefit when traders resell their produce at higher prices.

Q74. What is wallet balance stored in?

Answer: users.walletBalance column in PostgreSQL.

Q75. What is checkout_order?

Answer: Supabase RPC that atomically processes purchase, payments, royalties, and stock updates.

Q76. What is AgroElevate?

Answer: A React Native Android agricultural marketplace connecting farmers, traders, customers, industrialists, and admins with wallet payments and automatic farmer royalty on resale.

Q77. What framework did you use?

Answer: React Native with Expo SDK 54 and TypeScript.

Q78. What is your backend?

Answer: Supabase — PostgreSQL database, authentication, edge functions, and RPC stored procedures.

Q79. How many user roles exist?

Answer: Five: Farmer, Trader, Customer, Industrialist, Admin.

Q80. What is React Native?

Answer: A cross-platform framework to build mobile apps using JavaScript/React that renders native UI components.

Q81. What is Expo?

Answer: A toolchain and platform on top of React Native for easier development, native module access, and cloud builds (EAS).

Q82. What is TypeScript?

Answer: A typed superset of JavaScript that compiles to JavaScript, catching errors at compile time.

Q83. What database do you use?

Answer: PostgreSQL hosted on Supabase.

Q84. What payment gateway?

Answer: Razorpay for wallet top-up.

Q85. What is the royalty percentage?

Answer: 12.5% of profit per unit on trader resale, paid to the original farmer.

Q86. What is JWT?

Answer: JSON Web Token — a signed token proving authentication, issued by Supabase Auth.

Q87. What is AsyncStorage?

Answer: Local persistent key-value storage on the device used for session caching.

Q88. What is Expo Router?

Answer: File-based routing where files in app/ directory map to navigation routes.

Q89. What is an APK?

Answer: Android Package — installable application file.

Q90. What is Gradle?

Answer: Android build automation tool that compiles and packages the app.

Q91. What is Metro?

Answer: The JavaScript bundler for React Native.

Q92. What is Supabase Auth?

Answer: Authentication service supporting email/password, issuing sessions and JWTs.

Q93. What is a Primary Key?

Answer: A column that uniquely identifies each row in a table.

Q94. What is a Foreign Key?

Answer: A column referencing a primary key in another table to enforce relationships.

Q95. What is RPC in Supabase?

Answer: Remote Procedure Call — invoking PostgreSQL functions from the client via API.

Q96. What is EAS Build?

Answer: Expo Application Services cloud build system for generating APK/AAB.

Q97. What is the package name?

Answer: com.agronex.farmconnect

Q98. What problem does royalty solve?

Answer: Ensures original farmers benefit when traders resell their produce at higher prices.

Q99. What is wallet balance stored in?

Answer: users.walletBalance column in PostgreSQL.

Q100. What is checkout_order?

Answer: Supabase RPC that atomically processes purchase, payments, royalties, and stock updates.

23.2 — Intermediate Questions (100 Questions)

Q101. Explain the authentication flow step by step.

Answer: signInWithPassword → JWT session in AsyncStorage → onAuthStateChanged → ensureUserRecords creates profile/users → refreshUserData → index.tsx routes by role.

Q102. How does Expo Router protect routes?

Answer: index.tsx checks session; unauthenticated users redirect to /auth/login. Role-specific layouts only reachable after auth.

Q103. Difference between profiles and users tables?

Answer: profiles links to Supabase Auth for identity/admin; users holds wallet balance and commerce fields used by checkout RPC.

Q104. How is commerce metadata stored?

Answer: JSON string in products.description parsed by parseCommerceMeta — contains original_farmer_id, product_kind, royalty_percent, ownership_chain.

Q105. How does FlatList differ from ScrollView?

Answer: FlatList virtualizes items — only renders visible rows, better for long marketplace lists.

Q106. What is useFocusEffect?

Answer: Hook that runs callback when screen gains focus — used to refetch wallet/orders on tab switch.

Q107. How does Razorpay integration work?

Answer: Edge function creates order → native SDK opens checkout → webhook confirms payment → add_funds RPC credits wallet → client polls payment_intents.

Q108. What is SECURITY DEFINER on RPC?

Answer: Function runs with owner privileges, not caller — needed for atomic wallet updates but must validate auth inside.

Q109. Why React Context for auth?

Answer: Avoids prop drilling — any screen can use useAuth() for userData and signOut.

Q110. How are seller orders fetched?

Answer: Query order_items by farmerId/sellerId, collect orderIds, fetch orders with items joined.

Q111. What is product_kind trader_relist?

Answer: Indicates a trader reselling previously purchased produce; triggers royalty calculation on sale.

Q112. How is profit calculated for royalty?

Answer: profit_per_unit = sale pricePerUnit - original purchase_price_per_unit; royalty = profit × quantity × 12.5%.

Q113. What happens if wallet balance is insufficient?

Answer: checkout_order RPC raises exception "Insufficient wallet balance" — order not created.

Q114. What is the new architecture flag?

Answer: newArchEnabled: true in app.json enables React Native New Architecture (Fabric/TurboModules).

Q115. How do you handle DB column naming mismatch?

Answer: Normalize in API helpers — accept both camelCase and snake_case (orderCreatedAt, walletBalance/wallet_balance).

Q116. What is an Edge Function?

Answer: Serverless Deno function deployed on Supabase for secrets (Razorpay keys) and webhooks.

Q117. Explain order status lifecycle.

Answer: pending → accepted → shipped → delivered. Updated by seller or admin.

Q118. Why hide relistings from farmer marketplace?

Answer: Farmers browse source produce to buy/resell context; relistings are trader products — filtered when role=farmer.

Q119. What is withRetry in asyncUtils?

Answer: Retries failed network requests with backoff for flaky mobile connections.

Q120. How does AI pricing assistant work?

Answer: fetchCropPricing calls backend/edge API with crop name and location; suggests price when farmer adds crop.

Q121. What permissions does the app request?

Answer: Camera, storage, fine/coarse location — for crop photos and market intelligence.

Q122. Difference between APK and AAB build profiles?

Answer: preview profile in eas.json builds APK for testing; production builds AAB for Play Store.

Q123. What is normalizeAppRole?

Answer: Maps DB role middleman to app role trader for routing consistency.

Q124. How is stock decremented?

Answer: checkout_order updates products.quantity or soldQuantity; status sold when depleted.

Q125. What is payment_intents table for?

Answer: Tracks Razorpay top-up lifecycle: pending until webhook marks paid.

Q126. Explain the authentication flow step by step.

Answer: signInWithPassword → JWT session in AsyncStorage → onAuthStateChanged → ensureUserRecords creates profile/users → refreshUserData → index.tsx routes by role.

Q127. How does Expo Router protect routes?

Answer: index.tsx checks session; unauthenticated users redirect to /auth/login. Role-specific layouts only reachable after auth.

Q128. Difference between profiles and users tables?

Answer: profiles links to Supabase Auth for identity/admin; users holds wallet balance and commerce fields used by checkout RPC.

Q129. How is commerce metadata stored?

Answer: JSON string in products.description parsed by parseCommerceMeta — contains original_farmer_id, product_kind, royalty_percent, ownership_chain.

Q130. How does FlatList differ from ScrollView?

Answer: FlatList virtualizes items — only renders visible rows, better for long marketplace lists.

Q131. What is useFocusEffect?

Answer: Hook that runs callback when screen gains focus — used to refetch wallet/orders on tab switch.

Q132. How does Razorpay integration work?

Answer: Edge function creates order → native SDK opens checkout → webhook confirms payment → add_funds RPC credits wallet → client polls payment_intents.

Q133. What is SECURITY DEFINER on RPC?

Answer: Function runs with owner privileges, not caller — needed for atomic wallet updates but must validate auth inside.

Q134. Why React Context for auth?

Answer: Avoids prop drilling — any screen can use useAuth() for userData and signOut.

Q135. How are seller orders fetched?

Answer: Query order_items by farmerId/sellerId, collect orderIds, fetch orders with items joined.

Q136. What is product_kind trader_relist?

Answer: Indicates a trader reselling previously purchased produce; triggers royalty calculation on sale.

Q137. How is profit calculated for royalty?

Answer: profit_per_unit = sale pricePerUnit - original purchase_price_per_unit; royalty = profit × quantity × 12.5%.

Q138. What happens if wallet balance is insufficient?

Answer: checkout_order RPC raises exception "Insufficient wallet balance" — order not created.

Q139. What is the new architecture flag?

Answer: newArchEnabled: true in app.json enables React Native New Architecture (Fabric/TurboModules).

Q140. How do you handle DB column naming mismatch?

Answer: Normalize in API helpers — accept both camelCase and snake_case (orderCreatedAt, walletBalance/wallet_balance).

Q141. What is an Edge Function?

Answer: Serverless Deno function deployed on Supabase for secrets (Razorpay keys) and webhooks.

Q142. Explain order status lifecycle.

Answer: pending → accepted → shipped → delivered. Updated by seller or admin.

Q143. Why hide relistings from farmer marketplace?

Answer: Farmers browse source produce to buy/resell context; relistings are trader products — filtered when role=farmer.

Q144. What is withRetry in asyncUtils?

Answer: Retries failed network requests with backoff for flaky mobile connections.

Q145. How does AI pricing assistant work?

Answer: fetchCropPricing calls backend/edge API with crop name and location; suggests price when farmer adds crop.

Q146. What permissions does the app request?

Answer: Camera, storage, fine/coarse location — for crop photos and market intelligence.

Q147. Difference between APK and AAB build profiles?

Answer: preview profile in eas.json builds APK for testing; production builds AAB for Play Store.

Q148. What is normalizeAppRole?

Answer: Maps DB role middleman to app role trader for routing consistency.

Q149. How is stock decremented?

Answer: checkout_order updates products.quantity or soldQuantity; status sold when depleted.

Q150. What is payment_intents table for?

Answer: Tracks Razorpay top-up lifecycle: pending until webhook marks paid.

Q151. Explain the authentication flow step by step.

Answer: signInWithPassword → JWT session in AsyncStorage → onAuthStateChanged → ensureUserRecords creates profile/users → refreshUserData → index.tsx routes by role.

Q152. How does Expo Router protect routes?

Answer: index.tsx checks session; unauthenticated users redirect to /auth/login. Role-specific layouts only reachable after auth.

Q153. Difference between profiles and users tables?

Answer: profiles links to Supabase Auth for identity/admin; users holds wallet balance and commerce fields used by checkout RPC.

Q154. How is commerce metadata stored?

Answer: JSON string in products.description parsed by parseCommerceMeta — contains original_farmer_id, product_kind, royalty_percent, ownership_chain.

Q155. How does FlatList differ from ScrollView?

Answer: FlatList virtualizes items — only renders visible rows, better for long marketplace lists.

Q156. What is useFocusEffect?

Answer: Hook that runs callback when screen gains focus — used to refetch wallet/orders on tab switch.

Q157. How does Razorpay integration work?

Answer: Edge function creates order → native SDK opens checkout → webhook confirms payment → add_funds RPC credits wallet → client polls payment_intents.

Q158. What is SECURITY DEFINER on RPC?

Answer: Function runs with owner privileges, not caller — needed for atomic wallet updates but must validate auth inside.

Q159. Why React Context for auth?

Answer: Avoids prop drilling — any screen can use useAuth() for userData and signOut.

Q160. How are seller orders fetched?

Answer: Query order_items by farmerId/sellerId, collect orderIds, fetch orders with items joined.

Q161. What is product_kind trader_relist?

Answer: Indicates a trader reselling previously purchased produce; triggers royalty calculation on sale.

Q162. How is profit calculated for royalty?

Answer: $\text{profit_per_unit} = \text{sale pricePerUnit} - \text{original purchase_price_per_unit}$; $\text{royalty} = \text{profit} \times \text{quantity} \times 12.5\%$.

Q163. What happens if wallet balance is insufficient?

Answer: checkout_order RPC raises exception "Insufficient wallet balance" — order not created.

Q164. What is the new architecture flag?

Answer: newArchEnabled: true in app.json enables React Native New Architecture (Fabric/TurboModules).

Q165. How do you handle DB column naming mismatch?

Answer: Normalize in API helpers — accept both camelCase and snake_case (orderCreatedAt, walletBalance/wallet_balance).

Q166. What is an Edge Function?

Answer: Serverless Deno function deployed on Supabase for secrets (Razorpay keys) and webhooks.

Q167. Explain order status lifecycle.

Answer: pending → accepted → shipped → delivered. Updated by seller or admin.

Q168. Why hide relistings from farmer marketplace?

Answer: Farmers browse source produce to buy/resell context; relistings are trader products — filtered when role=farmer.

Q169. What is withRetry in asyncUtils?

Answer: Retries failed network requests with backoff for flaky mobile connections.

Q170. How does AI pricing assistant work?

Answer: fetchCropPricing calls backend/edge API with crop name and location; suggests price when farmer adds crop.

Q171. What permissions does the app request?

Answer: Camera, storage, fine/coarse location — for crop photos and market intelligence.

Q172. Difference between APK and AAB build profiles?

Answer: preview profile in eas.json builds APK for testing; production builds AAB for Play Store.

Q173. What is normalizeAppRole?

Answer: Maps DB role middleman to app role trader for routing consistency.

Q174. How is stock decremented?

Answer: checkout_order updates products.quantity or soldQuantity; status sold when depleted.

Q175. What is payment_intents table for?

Answer: Tracks Razorpay top-up lifecycle: pending until webhook marks paid.

Q176. Explain the authentication flow step by step.

Answer: signInWithPassword → JWT session in AsyncStorage → onAuthStateChanged → ensureUserRecords creates profile/users → refreshUserData → index.tsx routes by role.

Q177. How does Expo Router protect routes?

Answer: index.tsx checks session; unauthenticated users redirect to /auth/login. Role-specific layouts only reachable after auth.

Q178. Difference between profiles and users tables?

Answer: profiles links to Supabase Auth for identity/admin; users holds wallet balance and commerce fields used by checkout RPC.

Q179. How is commerce metadata stored?

Answer: JSON string in products.description parsed by parseCommerceMeta — contains original_farmer_id, product_kind, royalty_percent, ownership_chain.

Q180. How does FlatList differ from ScrollView?

Answer: FlatList virtualizes items — only renders visible rows, better for long marketplace lists.

Q181. What is useFocusEffect?

Answer: Hook that runs callback when screen gains focus — used to refetch wallet/orders on tab switch.

Q182. How does Razorpay integration work?

Answer: Edge function creates order → native SDK opens checkout → webhook confirms payment → add_funds RPC credits wallet → client polls payment_intents.

Q183. What is SECURITY DEFINER on RPC?

Answer: Function runs with owner privileges, not caller — needed for atomic wallet updates but must validate auth inside.

Q184. Why React Context for auth?

Answer: Avoids prop drilling — any screen can use useAuth() for userData and signOut.

Q185. How are seller orders fetched?

Answer: Query order_items by farmerId/sellerId, collect orderIds, fetch orders with items joined.

Q186. What is product_kind trader_relist?

Answer: Indicates a trader reselling previously purchased produce; triggers royalty calculation on sale.

Q187. How is profit calculated for royalty?

Answer: $\text{profit_per_unit} = \text{sale pricePerUnit} - \text{original purchase_price_per_unit}$; $\text{royalty} = \text{profit} \times \text{quantity} \times 12.5\%$.

Q188. What happens if wallet balance is insufficient?

Answer: `checkout_order` RPC raises exception "Insufficient wallet balance" — order not created.

Q189. What is the new architecture flag?

Answer: `newArchEnabled`: true in `app.json` enables React Native New Architecture (Fabric/TurboModules).

Q190. How do you handle DB column naming mismatch?

Answer: Normalize in API helpers — accept both `camelCase` and `snake_case` (`orderCreatedAt`, `walletBalance/wallet_balance`).

Q191. What is an Edge Function?

Answer: Serverless Deno function deployed on Supabase for secrets (Razorpay keys) and webhooks.

Q192. Explain order status lifecycle.

Answer: `pending` → `accepted` → `shipped` → `delivered`. Updated by seller or admin.

Q193. Why hide relistings from farmer marketplace?

Answer: Farmers browse source produce to buy/resell context; relistings are trader products — filtered when `role=farmer`.

Q194. What is `withRetry` in `asyncUtils`?

Answer: Retries failed network requests with backoff for flaky mobile connections.

Q195. How does AI pricing assistant work?

Answer: `fetchCropPricing` calls backend/edge API with crop name and location; suggests price when farmer adds crop.

Q196. What permissions does the app request?

Answer: Camera, storage, fine/coarse location — for crop photos and market intelligence.

Q197. Difference between APK and AAB build profiles?

Answer: preview profile in `eas.json` builds APK for testing; production builds AAB for Play Store.

Q198. What is `normalizeAppRole`?

Answer: Maps DB role `middleman` to app role `trader` for routing consistency.

Q199. How is stock decremented?

Answer: `checkout_order` updates `products.quantity` or `soldQuantity`; status `sold` when depleted.

Q200. What is `payment_intents` table for?

Answer: Tracks Razorpay top-up lifecycle: `pending` until webhook marks paid.

23.3 — Advanced Questions (100 Questions)

Q201. Why use RPC instead of client-side multi-step checkout?

Answer: Atomicity — all wallet debits, credits, royalty, and stock updates succeed or fail together, preventing partial inconsistent state.

Q202. Race condition if two users buy last unit?

Answer: PostgreSQL transaction in RPC locks rows; second checkout fails stock validation.

Q203. How would you enable RLS properly?

Answer: Policies: users read own wallet_history; sellers see their order_items; admin role bypass via service role or claims.

Q204. JWT refresh flow?

Answer: Supabase client autoRefreshToken exchanges refresh token before access token expiry; stored in AsyncStorage.

Q205. Webhook security for Razorpay?

Answer: Verify HMAC signature with webhook secret in Edge Function before crediting wallet.

Q206. Idempotency of payment webhook?

Answer: Check payment_intents status; if already paid, skip duplicate add_funds.

Q207. Scaling marketplace queries?

Answer: Indexes on products(quantity), products(seller_id), order_items(farmerId); pagination with range().

Q208. Why JSON in description vs separate columns?

Answer: Flexible commerce meta without schema migration for new royalty fields; parsed at runtime.

Q209. React Native bridge performance?

Answer: New Architecture reduces bridge overhead; FlatList virtualization reduces render load.

Q210. Memory leak prevention in useEffect?

Answer: Return cleanup functions, mounted flag in AuthContext, unsubscribe auth listener.

Q211. Testing checkout_order without real money?

Answer: Admin add_funds for test wallet; runtime-qa.mjs dry-runs RPC existence.

Q212. Migrating from MongoDB/Firebase?

Answer: Project evolved to Supabase PostgreSQL for relational integrity and RPC; AuthContext provisions profiles/users on login.

Q213. Ownership chain purpose?

Answer: Audit trail of who held produce: farmer → trader → industrialist for dispute resolution and royalty attribution.

Q214. Handling offline mode?

Answer: Currently requires network; could add AsyncStorage queue for actions with sync on reconnect.

Q215. Certificate pinning?

Answer: Not implemented; production hardening would pin Supabase/Razorpay TLS certs.

Q216. Multi-tenant isolation?

Answer: Single platform; user_id scoping on all queries and RLS.

Q217. GDPR/data deletion?

Answer: Would cascade delete auth user, profiles, anonymize orders — not fully implemented.

Q218. Optimistic UI for buy button?

Answer: Currently waits for RPC; could show pending state then rollback on error.

Q219. Deep linking to crop-details?

Answer: Expo scheme agroelevate:// crop-details?id=uuid via expo-linking.

Q220. Hermes engine?

Answer: Default JS engine on Android in RN 0.81 — faster startup, lower memory.

Q221. Proguard/R8 in release?

Answer: Shrinks and obfuscates Java bytecode in release Android builds.

Q222. Concurrent wallet top-up and purchase?

Answer: PostgreSQL row lock on users.walletBalance serializes updates.

Q223. Extending royalty to industrial processed goods?

Answer: product_kind processed in commerceMeta with processed_product_id linkage.

Q224. CI/CD pipeline?

Answer: EAS Build on git push; runtime-qa.mjs validates backend connectivity.

Q225. Monitoring production errors?

Answer: Could add Sentry; ErrorBoundary catches React render errors in _layout.tsx.

Q226. Why use RPC instead of client-side multi-step checkout?

Answer: Atomicity — all wallet debits, credits, royalty, and stock updates succeed or fail together, preventing partial inconsistent state.

Q227. Race condition if two users buy last unit?

Answer: PostgreSQL transaction in RPC locks rows; second checkout fails stock validation.

Q228. How would you enable RLS properly?

Answer: Policies: users read own wallet_history; sellers see their order_items; admin role bypass via service role or claims.

Q229. JWT refresh flow?

Answer: Supabase client autoRefreshToken exchanges refresh token before access token expiry; stored in AsyncStorage.

Q230. Webhook security for Razorpay?

Answer: Verify HMAC signature with webhook secret in Edge Function before crediting wallet.

Q231. Idempotency of payment webhook?

Answer: Check payment_intents status; if already paid, skip duplicate add_funds.

Q232. Scaling marketplace queries?

Answer: Indexes on products(quantity), products(seller_id), order_items(farmerId); pagination with range().

Q233. Why JSON in description vs separate columns?

Answer: Flexible commerce meta without schema migration for new royalty fields; parsed at runtime.

Q234. React Native bridge performance?

Answer: New Architecture reduces bridge overhead; FlatList virtualization reduces render load.

Q235. Memory leak prevention in useEffect?

Answer: Return cleanup functions, mounted flag in AuthContext, unsubscribe auth listener.

Q236. Testing checkout_order without real money?

Answer: Admin add_funds for test wallet; runtime-qa.mjs dry-runs RPC existence.

Q237. Migrating from MongoDB/Firebase?

Answer: Project evolved to Supabase PostgreSQL for relational integrity and RPC; AuthContext provisions profiles/users on login.

Q238. Ownership chain purpose?

Answer: Audit trail of who held produce: farmer → trader → industrialist for dispute resolution and royalty attribution.

Q239. Handling offline mode?

Answer: Currently requires network; could add AsyncStorage queue for actions with sync on reconnect.

Q240. Certificate pinning?

Answer: Not implemented; production hardening would pin Supabase/Razorpay TLS certs.

Q241. Multi-tenant isolation?

Answer: Single platform; user_id scoping on all queries and RLS.

Q242. GDPR/data deletion?

Answer: Would cascade delete auth user, profiles, anonymize orders — not fully implemented.

Q243. Optimistic UI for buy button?

Answer: Currently waits for RPC; could show pending state then rollback on error.

Q244. Deep linking to crop-details?

Answer: Expo scheme agroelevate:// crop-details?id=uuid via expo-linking.

Q245. Hermes engine?

Answer: Default JS engine on Android in RN 0.81 — faster startup, lower memory.

Q246. Proguard/R8 in release?

Answer: Shrinks and obfuscates Java bytecode in release Android builds.

Q247. Concurrent wallet top-up and purchase?

Answer: PostgreSQL row lock on users.walletBalance serializes updates.

Q248. Extending royalty to industrial processed goods?

Answer: product_kind processed in commerceMeta with processed_product_id linkage.

Q249. CI/CD pipeline?

Answer: EAS Build on git push; runtime-qa.mjs validates backend connectivity.

Q250. Monitoring production errors?

Answer: Could add Sentry; ErrorBoundary catches React render errors in _layout.tsx.

Q251. Why use RPC instead of client-side multi-step checkout?

Answer: Atomicity — all wallet debits, credits, royalty, and stock updates succeed or fail together, preventing partial inconsistent state.

Q252. Race condition if two users buy last unit?

Answer: PostgreSQL transaction in RPC locks rows; second checkout fails stock validation.

Q253. How would you enable RLS properly?

Answer: Policies: users read own wallet_history; sellers see their order_items; admin role bypass via service role or claims.

Q254. JWT refresh flow?

Answer: Supabase client `autoRefreshToken` exchanges refresh token before access token expiry; stored in `AsyncStorage`.

Q255. Webhook security for Razorpay?

Answer: Verify HMAC signature with webhook secret in Edge Function before crediting wallet.

Q256. Idempotency of payment webhook?

Answer: Check `payment_intents` status; if already paid, skip duplicate `add_funds`.

Q257. Scaling marketplace queries?

Answer: Indexes on `products(quantity)`, `products(seller_id)`, `order_items(farmerId)`; pagination with `range()`.

Q258. Why JSON in description vs separate columns?

Answer: Flexible commerce meta without schema migration for new royalty fields; parsed at runtime.

Q259. React Native bridge performance?

Answer: New Architecture reduces bridge overhead; `FlatList` virtualization reduces render load.

Q260. Memory leak prevention in `useEffect`?

Answer: Return cleanup functions, mounted flag in `AuthContext`, unsubscribe auth listener.

Q261. Testing `checkout_order` without real money?

Answer: Admin `add_funds` for test wallet; `runtime-qa.mjs` dry-runs RPC existence.

Q262. Migrating from MongoDB/Firebase?

Answer: Project evolved to Supabase PostgreSQL for relational integrity and RPC; `AuthContext` provisions profiles/users on login.

Q263. Ownership chain purpose?

Answer: Audit trail of who held produce: farmer → trader → industrialist for dispute resolution and royalty attribution.

Q264. Handling offline mode?

Answer: Currently requires network; could add `AsyncStorage` queue for actions with sync on reconnect.

Q265. Certificate pinning?

Answer: Not implemented; production hardening would pin Supabase/Razorpay TLS certs.

Q266. Multi-tenant isolation?

Answer: Single platform; `user_id` scoping on all queries and RLS.

Q267. GDPR/data deletion?

Answer: Would cascade delete auth user, profiles, anonymize orders — not fully implemented.

Q268. Optimistic UI for buy button?

Answer: Currently waits for RPC; could show pending state then rollback on error.

Q269. Deep linking to crop-details?

Answer: Expo scheme `agroelevate://crop-details?id=uuid` via expo-linking.

Q270. Hermes engine?

Answer: Default JS engine on Android in RN 0.81 — faster startup, lower memory.

Q271. Proguard/R8 in release?

Answer: Shrinks and obfuscates Java bytecode in release Android builds.

Q272. Concurrent wallet top-up and purchase?

Answer: PostgreSQL row lock on users.walletBalance serializes updates.

Q273. Extending royalty to industrial processed goods?

Answer: product_kind processed in commerceMeta with processed_product_id linkage.

Q274. CI/CD pipeline?

Answer: EAS Build on git push; runtime-qa.mjs validates backend connectivity.

Q275. Monitoring production errors?

Answer: Could add Sentry; ErrorBoundary catches React render errors in _layout.tsx.

Q276. Why use RPC instead of client-side multi-step checkout?

Answer: Atomicity — all wallet debits, credits, royalty, and stock updates succeed or fail together, preventing partial inconsistent state.

Q277. Race condition if two users buy last unit?

Answer: PostgreSQL transaction in RPC locks rows; second checkout fails stock validation.

Q278. How would you enable RLS properly?

Answer: Policies: users read own wallet_history; sellers see their order_items; admin role bypass via service role or claims.

Q279. JWT refresh flow?

Answer: Supabase client autoRefreshToken exchanges refresh token before access token expiry; stored in AsyncStorage.

Q280. Webhook security for Razorpay?

Answer: Verify HMAC signature with webhook secret in Edge Function before crediting wallet.

Q281. Idempotency of payment webhook?

Answer: Check payment_intents status; if already paid, skip duplicate add_funds.

Q282. Scaling marketplace queries?

Answer: Indexes on products(quantity), products(seller_id), order_items(farmerId); pagination with range().

Q283. Why JSON in description vs separate columns?

Answer: Flexible commerce meta without schema migration for new royalty fields; parsed at runtime.

Q284. React Native bridge performance?

Answer: New Architecture reduces bridge overhead; FlatList virtualization reduces render load.

Q285. Memory leak prevention in useEffect?

Answer: Return cleanup functions, mounted flag in AuthContext, unsubscribe auth listener.

Q286. Testing checkout_order without real money?

Answer: Admin add_funds for test wallet; runtime-qa.mjs dry-runs RPC existence.

Q287. Migrating from MongoDB/Firebase?

Answer: Project evolved to Supabase PostgreSQL for relational integrity and RPC; AuthContext provisions profiles/users on login.

Q288. Ownership chain purpose?

Answer: Audit trail of who held produce: farmer → trader → industrialist for dispute resolution and royalty attribution.

Q289. Handling offline mode?

Answer: Currently requires network; could add AsyncStorage queue for actions with sync on reconnect.

Q290. Certificate pinning?

Answer: Not implemented; production hardening would pin Supabase/Razorpay TLS certs.

Q291. Multi-tenant isolation?

Answer: Single platform; user_id scoping on all queries and RLS.

Q292. GDPR/data deletion?

Answer: Would cascade delete auth user, profiles, anonymize orders — not fully implemented.

Q293. Optimistic UI for buy button?

Answer: Currently waits for RPC; could show pending state then rollback on error.

Q294. Deep linking to crop-details?

Answer: Expo scheme agroelevate:// crop-details?id=uuid via expo-linking.

Q295. Hermes engine?

Answer: Default JS engine on Android in RN 0.81 — faster startup, lower memory.

Q296. Proguard/R8 in release?

Answer: Shrinks and obfuscates Java bytecode in release Android builds.

Q297. Concurrent wallet top-up and purchase?

Answer: PostgreSQL row lock on users.walletBalance serializes updates.

Q298. Extending royalty to industrial processed goods?

Answer: product_kind processed in commerceMeta with processed_product_id linkage.

Q299. CI/CD pipeline?

Answer: EAS Build on git push; runtime-qa.mjs validates backend connectivity.

Q300. Monitoring production errors?

Answer: Could add Sentry; ErrorBoundary catches React render errors in _layout.tsx.

23.4 — Project-Specific Questions (100 Questions)

Q301. What is the app name shown to users?

Answer: AgroElevate — slug agroelevate-app in Expo config.

Q302. What color is the primary brand?

Answer: #16a34a (green) — agriculture theme.

Q303. What tabs does farmer have?

Answer: Home, Market, Intel, Orders, Wallet, Profile — plus hidden add-crop and my-crops.

Q304. What is the intelligence module?

Answer: Market intelligence screens: live prices, MSP, forecast, benchmark, map, nearby markets, recommendations.

Q305. Which file creates Supabase client?

Answer: frontend/lib/supabase.ts

Q306. Which RPC adds wallet funds?

Answer: add_funds with p_user_id, p_amount, p_payment_method.

Q307. Maximum top-up amount?

Answer: ₹1,00,000 per transaction enforced in wallet.tsx.

Q308. What categories can farmers list?

Answer: Grains, Vegetables, Fruits, Pulses, Raw Materials.

Q309. Units supported for crops?

Answer: kg, quintal, ton, pieces.

Q310. How does farmer approval work?

Answer: Admin sets profiles.approved=true in approve-farmers screen.

Q311. What does suspended flag do?

Answer: profiles.suspended=true blocks or flags user — set on rejectFarmer.

Q312. Where is ErrorBoundary used?

Answer: Root _layout.tsx wrapping AuthProvider to catch render crashes.

Q313. What backend URL in app.json extra?

Answer: <http://10.0.2.2:8001> for Android emulator → host machine (AI pricing).

Q314. What is clampRoyaltyPercent?

Answer: Clamps royalty between 10% and 12.5% from metadata.

Q315. Default royalty percent constant?

Answer: DEFAULT_ROYALTY_PERCENT = 12.5 in commerceMeta.ts.

Q316. How does admin credit wallet?

Answer: adminApi.adminCreditWallet calls add_funds RPC with admin_credit method.

Q317. What files handle Razorpay on Android?

Answer: razorpayWallet.ts, razorpayCheckout.native.ts, wallet.tsx.

Q318. What is friendlyError?

Answer: asyncUtils helper mapping Supabase/PostgREST errors to user-readable messages.

Q319. CHUNK_SIZE in ordersApi?

Answer: 40 — batches order ID queries to avoid URL length limits.

Q320. What is isSellerRole?

Answer: Returns true for farmer, trader, industrialist — they receive sell-side orders.

Q321. What intelligence hook exists?

Answer: hooks/useIntelligence.ts for shared market data fetching.

Q322. Where is buildFarmerListingMeta used?

Answer: add-crop.tsx when inserting new product description.

Q323. What does runtime-qa verify?

Answer: checkout_order RPC exists, Supabase connectivity, schema health.

Q324. EAS project ID?

Answer: 5568e8e7-6cc1-4691-bb74-f2d4c09017a7 in app.json extra.eas.

Q325. What happens on provisioning failure?

Answer: index.tsx shows retry UI with retryProvisioning and sign out options.

Q326. What is the app name shown to users?

Answer: AgroElevate — slug agroelevate-app in Expo config.

Q327. What color is the primary brand?

Answer: #16a34a (green) — agriculture theme.

Q328. What tabs does farmer have?

Answer: Home, Market, Intel, Orders, Wallet, Profile — plus hidden add-crop and my-crops.

Q329. What is the intelligence module?

Answer: Market intelligence screens: live prices, MSP, forecast, benchmark, map, nearby markets, recommendations.

Q330. Which file creates Supabase client?

Answer: frontend/lib/supabase.ts

Q331. Which RPC adds wallet funds?

Answer: add_funds with p_user_id, p_amount, p_payment_method.

Q332. Maximum top-up amount?

Answer: ₹1,00,000 per transaction enforced in wallet.tsx.

Q333. What categories can farmers list?

Answer: Grains, Vegetables, Fruits, Pulses, Raw Materials.

Q334. Units supported for crops?

Answer: kg, quintal, ton, pieces.

Q335. How does farmer approval work?

Answer: Admin sets profiles.approved=true in approve-farmers screen.

Q336. What does suspended flag do?

Answer: profiles.suspended=true blocks or flags user — set on rejectFarmer.

Q337. Where is ErrorBoundary used?

Answer: Root _layout.tsx wrapping AuthProvider to catch render crashes.

Q338. What backend URL in app.json extra?

Answer: <http://10.0.2.2:8001> for Android emulator → host machine (AI pricing).

Q339. What is clampRoyaltyPercent?

Answer: Clamps royalty between 10% and 12.5% from metadata.

Q340. Default royalty percent constant?

Answer: DEFAULT_ROYALTY_PERCENT = 12.5 in commerceMeta.ts.

Q341. How does admin credit wallet?

Answer: adminApi.adminCreditWallet calls add_funds RPC with admin_credit method.

Q342. What files handle Razorpay on Android?

Answer: razorpayWallet.ts, razorpayCheckout.native.ts, wallet.tsx.

Q343. What is friendlyError?

Answer: asyncUtils helper mapping Supabase/PostgREST errors to user-readable messages.

Q344. CHUNK_SIZE in ordersApi?

Answer: 40 — batches order ID queries to avoid URL length limits.

Q345. What is isSellerRole?

Answer: Returns true for farmer, trader, industrialist — they receive sell-side orders.

Q346. What intelligence hook exists?

Answer: hooks/useIntelligence.ts for shared market data fetching.

Q347. Where is buildFarmerListingMeta used?

Answer: add-crop.tsx when inserting new product description.

Q348. What does runtime-qa verify?

Answer: checkout_order RPC exists, Supabase connectivity, schema health.

Q349. EAS project ID?

Answer: 5568e8e7-6cc1-4691-bb74-f2d4c09017a7 in app.json extra.eas.

Q350. What happens on provisioning failure?

Answer: index.tsx shows retry UI with retryProvisioning and sign out options.

Q351. What is the app name shown to users?

Answer: AgroElevate — slug agroelevate-app in Expo config.

Q352. What color is the primary brand?

Answer: #16a34a (green) — agriculture theme.

Q353. What tabs does farmer have?

Answer: Home, Market, Intel, Orders, Wallet, Profile — plus hidden add-crop and my-crops.

Q354. What is the intelligence module?

Answer: Market intelligence screens: live prices, MSP, forecast, benchmark, map, nearby markets, recommendations.

Q355. Which file creates Supabase client?

Answer: frontend/lib/supabase.ts

Q356. Which RPC adds wallet funds?

Answer: add_funds with p_user_id, p_amount, p_payment_method.

Q357. Maximum top-up amount?

Answer: ₹1,00,000 per transaction enforced in wallet.tsx.

Q358. What categories can farmers list?

Answer: Grains, Vegetables, Fruits, Pulses, Raw Materials.

Q359. Units supported for crops?

Answer: kg, quintal, ton, pieces.

Q360. How does farmer approval work?

Answer: Admin sets profiles.approved=true in approve-farmers screen.

Q361. What does suspended flag do?

Answer: profiles.suspended=true blocks or flags user — set on rejectFarmer.

Q362. Where is ErrorBoundary used?

Answer: Root _layout.tsx wrapping AuthProvider to catch render crashes.

Q363. What backend URL in app.json extra?

Answer: <http://10.0.2.2:8001> for Android emulator → host machine (AI pricing).

Q364. What is clampRoyaltyPercent?

Answer: Clamps royalty between 10% and 12.5% from metadata.

Q365. Default royalty percent constant?

Answer: DEFAULT_ROYALTY_PERCENT = 12.5 in commerceMeta.ts.

Q366. How does admin credit wallet?

Answer: adminApi.adminCreditWallet calls add_funds RPC with admin_credit method.

Q367. What files handle Razorpay on Android?

Answer: razorpayWallet.ts, razorpayCheckout.native.ts, wallet.tsx.

Q368. What is friendlyError?

Answer: asyncUtils helper mapping Supabase/PostgREST errors to user-readable messages.

Q369. CHUNK_SIZE in ordersApi?

Answer: 40 — batches order ID queries to avoid URL length limits.

Q370. What is isSellerRole?

Answer: Returns true for farmer, trader, industrialist — they receive sell-side orders.

Q371. What intelligence hook exists?

Answer: hooks/useIntelligence.ts for shared market data fetching.

Q372. Where is buildFarmerListingMeta used?

Answer: add-crop.tsx when inserting new product description.

Q373. What does runtime-qa verify?

Answer: checkout_order RPC exists, Supabase connectivity, schema health.

Q374. EAS project ID?

Answer: 5568e8e7-6cc1-4691-bb74-f2d4c09017a7 in app.json extra.eas.

Q375. What happens on provisioning failure?

Answer: index.tsx shows retry UI with retryProvisioning and sign out options.

Q376. What is the app name shown to users?

Answer: AgroElevate — slug agroelevate-app in Expo config.

Q377. What color is the primary brand?

Answer: #16a34a (green) — agriculture theme.

Q378. What tabs does farmer have?

Answer: Home, Market, Intel, Orders, Wallet, Profile — plus hidden add-crop and my-crops.

Q379. What is the intelligence module?

Answer: Market intelligence screens: live prices, MSP, forecast, benchmark, map, nearby markets, recommendations.

Q380. Which file creates Supabase client?

Answer: frontend/lib/supabase.ts

Q381. Which RPC adds wallet funds?

Answer: add_funds with p_user_id, p_amount, p_payment_method.

Q382. Maximum top-up amount?

Answer: ₹1,00,000 per transaction enforced in wallet.tsx.

Q383. What categories can farmers list?

Answer: Grains, Vegetables, Fruits, Pulses, Raw Materials.

Q384. Units supported for crops?

Answer: kg, quintal, ton, pieces.

Q385. How does farmer approval work?

Answer: Admin sets profiles.approved=true in approve-farmers screen.

Q386. What does suspended flag do?

Answer: profiles.suspended=true blocks or flags user — set on rejectFarmer.

Q387. Where is ErrorBoundary used?

Answer: Root _layout.tsx wrapping AuthProvider to catch render crashes.

Q388. What backend URL in app.json extra?

Answer: <http://10.0.2.2:8001> for Android emulator → host machine (AI pricing).

Q389. What is clampRoyaltyPercent?

Answer: Clamps royalty between 10% and 12.5% from metadata.

Q390. Default royalty percent constant?

Answer: DEFAULT_ROYALTY_PERCENT = 12.5 in commerceMeta.ts.

Q391. How does admin credit wallet?

Answer: adminApi.adminCreditWallet calls add_funds RPC with admin_credit method.

Q392. What files handle Razorpay on Android?

Answer: razorpayWallet.ts, razorpayCheckout.native.ts, wallet.tsx.

Q393. What is friendlyError?

Answer: asyncUtils helper mapping Supabase/PostgREST errors to user-readable messages.

Q394. CHUNK_SIZE in ordersApi?

Answer: 40 — batches order ID queries to avoid URL length limits.

Q395. What is isSellerRole?

Answer: Returns true for farmer, trader, industrialist — they receive sell-side orders.

Q396. What intelligence hook exists?

Answer: hooks/useIntelligence.ts for shared market data fetching.

Q397. Where is buildFarmerListingMeta used?

Answer: add-crop.tsx when inserting new product description.

Q398. What does runtime-qa verify?

Answer: checkout_order RPC exists, Supabase connectivity, schema health.

Q399. EAS project ID?

Answer: 5568e8e7-6cc1-4691-bb74-f2d4c09017a7 in app.json extra.eas.

Q400. What happens on provisioning failure?

Answer: index.tsx shows retry UI with retryProvisioning and sign out options.

23.5 — External Examiner Questions (100 Questions)

Q401. Why should farmers trust your royalty system?

Answer: Royalty is enforced in checkout_order database RPC, not app UI — cannot be skipped by modifying client.

Q402. Real-world deployment challenges?

Answer: Internet in rural areas, digital literacy, mandi regulations, payment KYC, logistics last mile.

Q403. How is your project different from eNAM?

Answer: eNAM is government mandi platform; AgroElevate adds wallet commerce, multi-role resale, and automated profit-sharing royalty.

Q404. Revenue model for platform?

Answer: Could charge commission on transactions, subscription for traders, or premium intelligence features.

Q405. What if trader sells at loss?

Answer: Royalty only applies when profit_per_unit > 0; no royalty on loss-making resale.

Q406. Socio-economic impact?

Answer: Increases farmer income transparency, reduces exploitation, digitizes agricultural trade.

Q407. Why mobile not web?

Answer: Farmers primarily use smartphones; mobile-first UX with large touch targets.

Q408. Data privacy for bank UPI?

Answer: Stored in profiles; should encrypt at rest and mask in UI for production.

Q409. Can admin misuse wallet credit?

Answer: Mitigate with audit logs, dual approval, and RLS restricting admin RPC to service role only.

Q410. Validate your tech stack choice in 2 minutes.

Answer: React Native: fast cross-platform. Supabase: SQL + auth + RPC for money logic. Razorpay: India-compliant payments. Expo: practical Android delivery for FYP timeline.

Q411. Single point of failure?

Answer: Supabase cloud dependency — mitigate with backups, monitoring, eventual multi-region.

Q412. How long did implementation take?

Answer: [Answer with your actual timeline — auth, marketplace, wallet, royalty, admin, intelligence phases.]

Q413. What was hardest bug?

Answer: [Prepare real example: e.g. wallet not updating until poll, role mapping middleman, checkout RPC migration.]

Q414. Unit testing status?

Answer: runtime-qa.mjs integration checks; manual role-based QA; unit tests can be added with Jest.

Q415. Demonstrate live — what will you show?

Answer: Register farmer → list crop → login trader → buy → relist → customer buys → show farmer royalty in wallet history.

Q416. IEEE/SDLC model followed?

Answer: Requirements → design (architecture, ERD) → implementation → testing (QA scripts) → deployment (EAS).

Q417. ER diagram entities?

Answer: users, profiles, products, orders, order_items, wallet_history, payment_intents, notifications — linked by FKs.

Q418. Non-functional requirements?

Answer: Usability for rural users, security for payments, scalability via Supabase, maintainability via TypeScript modular lib/.

Q419. What standards for agricultural data?

Answer: Could align with Agmark, FSSAI labeling for processed goods in future.

Q420. Environmental angle?

Answer: Reduces food waste via better demand matching; local produce lowers transport emissions.

Q421. If Supabase goes down?

Answer: App cannot function; need status page monitoring and cached read-only data for critical views.

Q422. Legal compliance for marketplace?

Answer: GST on trades, FSSAI for processed food, payment aggregator RBI guidelines via Razorpay.

Q423. Why 12.5% royalty specifically?

Answer: Configurable DEFAULT_ROYALTY_PERCENT; 12.5% balances farmer reward and trader margin (between 10-15% project spec).

Q424. Comparison with traditional mandi?

Answer: Mandi: physical, opaque pricing, high intermediary cut. AgroElevate: digital, recorded transactions, automatic royalty.

Q425. Your contribution vs team?

Answer: [Personalize — e.g. implemented checkout_order integration, wallet, intelligence module.]

Q426. Why should farmers trust your royalty system?

Answer: Royalty is enforced in checkout_order database RPC, not app UI — cannot be skipped by modifying client.

Q427. Real-world deployment challenges?

Answer: Internet in rural areas, digital literacy, mandi regulations, payment KYC, logistics last mile.

Q428. How is your project different from eNAM?

Answer: eNAM is government mandi platform; AgroElevate adds wallet commerce, multi-role resale, and automated profit-sharing royalty.

Q429. Revenue model for platform?

Answer: Could charge commission on transactions, subscription for traders, or premium intelligence features.

Q430. What if trader sells at loss?

Answer: Royalty only applies when profit_per_unit > 0; no royalty on loss-making resale.

Q431. Socio-economic impact?

Answer: Increases farmer income transparency, reduces exploitation, digitizes agricultural trade.

Q432. Why mobile not web?

Answer: Farmers primarily use smartphones; mobile-first UX with large touch targets.

Q433. Data privacy for bank UPI?

Answer: Stored in profiles; should encrypt at rest and mask in UI for production.

Q434. Can admin misuse wallet credit?

Answer: Mitigate with audit logs, dual approval, and RLS restricting admin RPC to service role only.

Q435. Validate your tech stack choice in 2 minutes.

Answer: React Native: fast cross-platform. Supabase: SQL + auth + RPC for money logic. Razorpay: India-compliant payments. Expo: practical Android delivery for FYP timeline.

Q436. Single point of failure?

Answer: Supabase cloud dependency — mitigate with backups, monitoring, eventual multi-region.

Q437. How long did implementation take?

Answer: [Answer with your actual timeline — auth, marketplace, wallet, royalty, admin, intelligence phases.]

Q438. What was hardest bug?

Answer: [Prepare real example: e.g. wallet not updating until poll, role mapping middleman, checkout RPC migration.]

Q439. Unit testing status?

Answer: runtime-qa.mjs integration checks; manual role-based QA; unit tests can be added with Jest.

Q440. Demonstrate live — what will you show?

Answer: Register farmer → list crop → login trader → buy → relist → customer buys → show farmer royalty in wallet history.

Q441. IEEE/SDLC model followed?

Answer: Requirements → design (architecture, ERD) → implementation → testing (QA scripts) → deployment (EAS).

Q442. ER diagram entities?

Answer: users, profiles, products, orders, order_items, wallet_history, payment_intents, notifications — linked by FKs.

Q443. Non-functional requirements?

Answer: Usability for rural users, security for payments, scalability via Supabase, maintainability via TypeScript modular lib/.

Q444. What standards for agricultural data?

Answer: Could align with Agmark, FSSAI labeling for processed goods in future.

Q445. Environmental angle?

Answer: Reduces food waste via better demand matching; local produce lowers transport emissions.

Q446. If Supabase goes down?

Answer: App cannot function; need status page monitoring and cached read-only data for critical views.

Q447. Legal compliance for marketplace?

Answer: GST on trades, FSSAI for processed food, payment aggregator RBI guidelines via Razorpay.

Q448. Why 12.5% royalty specifically?

Answer: Configurable DEFAULT_ROYALTY_PERCENT; 12.5% balances farmer reward and trader margin (between 10-15% project spec).

Q449. Comparison with traditional mandi?

Answer: Mandi: physical, opaque pricing, high intermediary cut. AgroElevate: digital, recorded transactions, automatic royalty.

Q450. Your contribution vs team?

Answer: [Personalize — e.g. implemented checkout_order integration, wallet, intelligence module.]

Q451. Why should farmers trust your royalty system?

Answer: Royalty is enforced in checkout_order database RPC, not app UI — cannot be skipped by modifying client.

Q452. Real-world deployment challenges?

Answer: Internet in rural areas, digital literacy, mandi regulations, payment KYC, logistics last mile.

Q453. How is your project different from eNAM?

Answer: eNAM is government mandi platform; AgroElevate adds wallet commerce, multi-role resale, and automated profit-sharing royalty.

Q454. Revenue model for platform?

Answer: Could charge commission on transactions, subscription for traders, or premium intelligence features.

Q455. What if trader sells at loss?

Answer: Royalty only applies when profit_per_unit > 0; no royalty on loss-making resale.

Q456. Socio-economic impact?

Answer: Increases farmer income transparency, reduces exploitation, digitizes agricultural trade.

Q457. Why mobile not web?

Answer: Farmers primarily use smartphones; mobile-first UX with large touch targets.

Q458. Data privacy for bank UPI?

Answer: Stored in profiles; should encrypt at rest and mask in UI for production.

Q459. Can admin misuse wallet credit?

Answer: Mitigate with audit logs, dual approval, and RLS restricting admin RPC to service role only.

Q460. Validate your tech stack choice in 2 minutes.

Answer: React Native: fast cross-platform. Supabase: SQL + auth + RPC for money logic. Razorpay: India-compliant payments. Expo: practical Android delivery for FYP timeline.

Q461. Single point of failure?

Answer: Supabase cloud dependency — mitigate with backups, monitoring, eventual multi-region.

Q462. How long did implementation take?

Answer: [Answer with your actual timeline — auth, marketplace, wallet, royalty, admin, intelligence phases.]

Q463. What was hardest bug?

Answer: [Prepare real example: e.g. wallet not updating until poll, role mapping middleman, checkout RPC migration.]

Q464. Unit testing status?

Answer: runtime-qa.mjs integration checks; manual role-based QA; unit tests can be added with Jest.

Q465. Demonstrate live — what will you show?

Answer: Register farmer → list crop → login trader → buy → relist → customer buys → show farmer royalty in wallet history.

Q466. IEEE/SDLC model followed?

Answer: Requirements → design (architecture, ERD) → implementation → testing (QA scripts) → deployment (EAS).

Q467. ER diagram entities?

Answer: users, profiles, products, orders, order_items, wallet_history, payment_intents, notifications — linked by FKs.

Q468. Non-functional requirements?

Answer: Usability for rural users, security for payments, scalability via Supabase, maintainability via TypeScript modular lib/.

Q469. What standards for agricultural data?

Answer: Could align with Agmark, FSSAI labeling for processed goods in future.

Q470. Environmental angle?

Answer: Reduces food waste via better demand matching; local produce lowers transport emissions.

Q471. If Supabase goes down?

Answer: App cannot function; need status page monitoring and cached read-only data for critical views.

Q472. Legal compliance for marketplace?

Answer: GST on trades, FSSAI for processed food, payment aggregator RBI guidelines via Razorpay.

Q473. Why 12.5% royalty specifically?

Answer: Configurable DEFAULT_ROYALTY_PERCENT; 12.5% balances farmer reward and trader margin (between 10-15% project spec).

Q474. Comparison with traditional mandi?

Answer: Mandi: physical, opaque pricing, high intermediary cut. AgroElevate: digital, recorded transactions, automatic royalty.

Q475. Your contribution vs team?

Answer: [Personalize — e.g. implemented checkout_order integration, wallet, intelligence module.]

Q476. Why should farmers trust your royalty system?

Answer: Royalty is enforced in checkout_order database RPC, not app UI — cannot be skipped by modifying client.

Q477. Real-world deployment challenges?

Answer: Internet in rural areas, digital literacy, mandi regulations, payment KYC, logistics last mile.

Q478. How is your project different from eNAM?

Answer: eNAM is government mandi platform; AgroElevate adds wallet commerce, multi-role resale, and automated profit-sharing royalty.

Q479. Revenue model for platform?

Answer: Could charge commission on transactions, subscription for traders, or premium intelligence features.

Q480. What if trader sells at loss?

Answer: Royalty only applies when profit_per_unit > 0; no royalty on loss-making resale.

Q481. Socio-economic impact?

Answer: Increases farmer income transparency, reduces exploitation, digitizes agricultural trade.

Q482. Why mobile not web?

Answer: Farmers primarily use smartphones; mobile-first UX with large touch targets.

Q483. Data privacy for bank UPI?

Answer: Stored in profiles; should encrypt at rest and mask in UI for production.

Q484. Can admin misuse wallet credit?

Answer: Mitigate with audit logs, dual approval, and RLS restricting admin RPC to service role only.

Q485. Validate your tech stack choice in 2 minutes.

Answer: React Native: fast cross-platform. Supabase: SQL + auth + RPC for money logic. Razorpay: India-compliant payments. Expo: practical Android delivery for FYP timeline.

Q486. Single point of failure?

Answer: Supabase cloud dependency — mitigate with backups, monitoring, eventual multi-region.

Q487. How long did implementation take?

Answer: [Answer with your actual timeline — auth, marketplace, wallet, royalty, admin, intelligence phases.]

Q488. What was hardest bug?

Answer: [Prepare real example: e.g. wallet not updating until poll, role mapping middleman, checkout RPC migration.]

Q489. Unit testing status?

Answer: runtime-qa.mjs integration checks; manual role-based QA; unit tests can be added with Jest.

Q490. Demonstrate live — what will you show?

Answer: Register farmer → list crop → login trader → buy → relist → customer buys → show farmer royalty in wallet history.

Q491. IEEE/SDLC model followed?

Answer: Requirements → design (architecture, ERD) → implementation → testing (QA scripts) → deployment (EAS).

Q492. ER diagram entities?

Answer: users, profiles, products, orders, order_items, wallet_history, payment_intents, notifications — linked by FKs.

Q493. Non-functional requirements?

Answer: Usability for rural users, security for payments, scalability via Supabase, maintainability via TypeScript modular lib/.

Q494. What standards for agricultural data?

Answer: Could align with Agmark, FSSAI labeling for processed goods in future.

Q495. Environmental angle?

Answer: Reduces food waste via better demand matching; local produce lowers transport emissions.

Q496. If Supabase goes down?

Answer: App cannot function; need status page monitoring and cached read-only data for critical views.

Q497. Legal compliance for marketplace?

Answer: GST on trades, FSSAI for processed food, payment aggregator RBI guidelines via Razorpay.

Q498. Why 12.5% royalty specifically?

Answer: Configurable DEFAULT_ROYALTY_PERCENT; 12.5% balances farmer reward and trader margin (between 10-15% project spec).

Q499. Comparison with traditional mandi?

Answer: Mandi: physical, opaque pricing, high intermediary cut. AgroElevate: digital, recorded transactions, automatic royalty.

Q500. Your contribution vs team?

Answer: [Personalize — e.g. implemented checkout_order integration, wallet, intelligence module.]

SECTION 24 — Cross Questions (Viva Practice Mode)

Instructions: In the viva, the examiner asks one question, waits for your answer, then drills deeper. Practice these chains:

1. "Explain authentication" → JWT → AsyncStorage → ensureUserRecords → profiles vs users
2. "How does payment work?" → Razorpay → Edge Function → webhook → add_funds → polling
3. "Explain royalty" → commerce meta → profit calculation → checkout_order → wallet_history type royalty
4. "Database design" → ERD → FK → RPC atomicity → why not NoSQL
5. "Android build" → Expo prebuild → Gradle → EAS → APK vs AAB → signing

SECTION 25 — Mock Viva (Strict Mode)

Sample opening: "Good morning. Tell me in 60 seconds what problem AgroElevate solves."

Expected: Indian farmers lose margin to middlemen. AgroElevate is a mobile marketplace with five roles, wallet payments via Razorpay, and automatic 12.5% royalty to original farmers when traders resell at profit — enforced in PostgreSQL RPC, not just the app.

Follow-up: "Show me in code where royalty is triggered." → crop-details.tsx calls checkout_order; commerceMeta.ts defines metadata; supabase_rpc.sql/create_order shows 12.5% calculation.

Follow-up: "What if I patch the APK to skip royalty?" → Server-side RPC still deducts and distributes; client cannot bypass database logic.

Follow-up: "Draw architecture." → Use diagram from Section 3.

Closing tip: Always tie answers to YOUR codebase files. Say "In our checkout_order RPC..." not generic textbook answers.

APPENDIX A — Quick Reference Commands

```
# Start development
cd frontend
npm install
npx expo start

# Run on Android emulator
npx expo run:android

# EAS cloud build (APK)
eas build --profile preview --platform android

# Production AAB
eas build --profile production --platform android
```

APPENDIX B — Environment Variables

EXPO_PUBLIC_SUPABASE_URL — Supabase project URL

EXPO_PUBLIC_SUPABASE_ANON_KEY — Public anon key (safe in client with RLS)

Razorpay secrets — Only in Supabase Edge Function environment

APPENDIX C — File Quick Index

Auth: contexts/AuthContext.tsx, app/auth/login.tsx, app/auth/register.tsx

Buy flow: app/crop-details.tsx

Royalty: lib/commerceMeta.ts, backend/supabase_rpc.sql

Wallet: app/farmer/wallet.tsx, lib/walletApi.ts, lib/razorpayWallet.ts

Orders: lib/ordersApi.ts, app/farmer/orders.tsx

Admin: lib/adminApi.ts, app/admin/dashboard.tsx

Config: app.json, eas.json, package.json
